ORCA Hand Firmware:
Feetech Bus, Motor Setup, and Calibration

MABEL Robotics — ORCA Hand Subsystem

June 22, 2026

Abstract

This document describes the low-level firmware for the MABEL ORCA hand: the 17-servo Feetech bus, the STS/SCS packet protocol, the one-time motor setup workflow, the tendon calibration procedure, and the file formats that feed the `orcahand` hardware layer. It is the reference for anyone assembling, re-tensioning, debugging, or porting the hand. The companion document (`docs/orcahand_api.pdf`) covers the Python/ROS API built on top of this.

Keywords: Feetech STS/SCS, serial bus servo, tendon-driven hand, motor calibration, end-stop detection, EEPROM configuration.

1 Hardware overview

The hand has **17 Feetech bus servos** on a single half-duplex TTL serial line at 1 Mbps:

- **16× HLS3915** — the finger actuators (IDs depend on the hand; see Table 1). Each finger has three tendon-driven joints; one abduction (`abd`) and two flexion joints (`mcp`, `pip`), except the thumb which adds `cmc`.
- **1× HLS3930** — the wrist actuator (higher torque).

All servos share one bus: commands are addressed by a 1-byte motor ID (1 . . . 17); ID `0xFE` is the broadcast address. The USB→TTL adapter is a CH340 (`1a86`) or CP210x (`10c4`); `orcahand.detect_port` and the firmware scripts auto-detect it by USB vendor ID, with a port override available.

Joint naming. Joints are {`finger`}_{`joint`}: `abd` (abduction), `mcp`/`pip` (finger flexion), and for the thumb `cmc`/`abd`/`mp`/`ip`. The wrist is `wrist`. The `mabel_description` URDF renames only the thumb chain (`cmc/mp/ip`→`mcp/pip/dip`); the ROS `joint_state_relay` performs that remap for RViz.

2 The STS/SCS packet protocol

Every transaction is a framed packet over the serial line:

```
0xFF 0xFF  ID  LEN  INST  [PARAM ...]  CHECKSUM
  LEN      = number of bytes after LEN  (INST + params + checksum)
  CHECKSUM = ~(ID + LEN + INST + params) & 0xFF
```

Table 1: Motor ID → joint map. The two hands are wired differently; this is the single source of truth (mirrored in `orcahand/configs/*_hand.yaml`).

	Left hand		**Right hand**
ID	joint	ID	joint
1	middle_pip	1	index_pip
2	pinky_mcp	2	index_mcp
3	pinky_pip	3	index_abd
4	pinky_abd	4	middle_abd
5	middle_abd	5	pinky_abd
6	index_abd	6	pinky_pip
7	index_mcp	7	pinky_mcp
8	index_pip	8	ring_pip
9	thumb_mp	9	middle_pip
10	thumb_ip	10	middle_mcp
11	thumb_abd	11	ring_pip
12	thumb_cmc	12	ring_mcp
13	ring_abd	13	pinky_abd
14	ring_pip	14	thumb_abd
15	ring_mcp	15	thumb_mp
16	middle_mcp	16	thumb_ip
17	wrist (HLS3930)	17	thumb_cmc

Table 2: Feetech STS/SCS control-table registers used by the firmware.

Addr	Name	Meaning
`0x28`	TORQUE_ENABLE	1 byte: 0 = free / back-drivable, 1 = hold
`0x29`	GOAL_ACC	1 byte: goal acceleration (S-curve)
`0x2A`	GOAL_POSITION	2 bytes: target tick (0–4095)
`0x2E`	GOAL_SPEED	2 bytes: speed cap (ticks/s; 0 = max)
`0x30`	TORQUE_LIMIT	2 bytes: torque ceiling (0–1000)
`0x38`	PRESENT_POSITION	start of 8-byte feedback block

The instruction byte selects the operation: `PING` (`0x01`), `READ` (`0x02`), `WRITE` (`0x03`), `SYNC_READ` (`0x82`), `SYNC_WRITE` (`0x83`). The implementation lives in one place — `orcahand/bus.py` (`FeetechBus`) — a dependency-free `pyserial` port that supersedes the ∼5 ad-hoc `Bus` classes that used to be copied across the firmware scripts.

Control table. Commands and feedback are register reads/writes on each servo's control table (Table 2). The hot path is `SYNC_WRITE` to `GOAL_POSITION`: a single broadcast packet carries a 2-byte goal for every motor, so the whole hand is commanded in one bus transaction. Feedback is an 8-byte block read starting at `PRESENT_POSITION`.

Feedback decoding. The 8-byte block at `0x38` is $[\text{pos}_2, \text{speed}_2, \text{load}_2, \text{volt}_1, \text{temp}_1]$. Position is a plain 12 bit little-endian value. Speed and load are *sign–magnitude*: speed's direction is bit 15, load's is bit 10. Voltage is in 0.1 V units; temperature in °C. These raw units are converted to SI one layer up (`orcahand.Joint`): position to radians via the calibrated affine map, speed to rad/s via $2\pi/4096$, load to a normalised effort $\ell/1000 \in [-1, 1]$.

3 One-time motor setup

Run once when assembling the hand or replacing a servo, from `firmware/orca_hand/feetech/`:

1. **Precheck** — `python setup/precheck.py` pings IDs 1 . . . 17, confirms the expected models (HLS3915 fingers, HLS3930 wrist), and prints a status table. Equivalent quick check post-install: `orcahand-scan --model right`.
2. **ID assignment** — fresh servos ship as ID 1. `python setup/setup_motor_chain.py` walks the power-cycle prompts to assign IDs 1 . . . 17; `--reset` reverts a servo to factory ID 1.
3. **EEPROM registers** — after calibration, `python setup/configure_registers.py` writes the per-servo angle limits and torque ceiling into EEPROM (a hardware backstop independent of the software clamp); `--read` verifies without writing.
4. **Latency** — `python setup/latency_test.py` measures bus round-trip time and the achievable loop rate (min/max/mean over N cycles).

4 Calibration

Calibration measures, for every motor, the raw tick end-stops of its joint's physical travel. These $[r^{\min}, r^{\max}]$ pairs are the hard safety clamp and the anchors of the angle→tick map in the API. Re-run after assembly, re-tensioning, or any mechanical change.

Procedure.

1. **Seat the tendons** — `python calibration/tension.py` runs a gentle jitter pass so cables settle before measurement (`--duration`, `--amplitude`).
2. **Sweep to end-stops** — `python calibration/calibrate_limits.py` drives each motor incrementally (5 ticks per 20 ms step) until a *stall* is detected, records the limit, backs off, and writes the result. Restrict to specific motors with `--ids 7 8 9`.
3. **Faster re-runs** — `smart_calibration.py` parallelises the sweep once rough limits exist; `manual_calibration.py` is an interactive GUI to capture limits by hand.

Stall detection. A joint has reached its stop when the measured position stops changing under continued command. Over a sliding window of N_{stable}=10 samples, the limit is declared when

$$\max_i r_i - \min_i r_i < \epsilon_{\text{stall}} \ (\approx 7 \text{ ticks}), \tag{1}$$

after which the script backs off and stores $r^{\text{center}} = \frac{1}{2}(r^{\min} + r^{\max})$. Multiple joints are swept together via `SYNC_WRITE`/`SYNC_READ` to keep the procedure quick.

Output files. Calibration writes two artifacts under `data/calibration/`:

```
// joint_limits.json  -- the per-motor tick end-stops (consumed everywhere)
{ "7": { "joint": "index_mcp", "min": 1885, "max": 2910,
         "center": 2397, "range": 1025 }, ... }
```

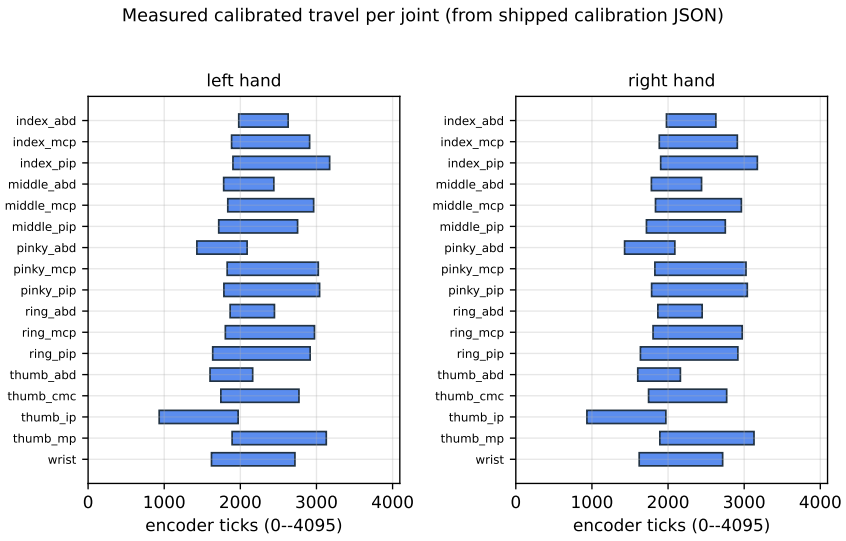


Figure 1: Measured calibrated travel per joint (encoder ticks), read from the shipped `left/right_calibration.json`. Regenerated by `docs/make_figures.py`.

```
# calibration.yaml -- metadata + limits in radians (provenance / audit)
calibrated: true
wrist_calibrated: false
motor_limits: { 12: [0.0798, 0.4694], ... }
```

The ROS package and the `orcahand` package consume the same `joint_limits.json` format (shipped as `<side>_calibration.json`). Fig. 1 shows the measured travel of every joint on both hands, straight from these files: abduction joints have the narrowest spans (∼560 ticks to 660 ticks), flexion joints the widest (∼1000 ticks to 1270 ticks).

5 Control and demo scripts

The `feetech/control/` folder holds workbench tools. All load `data/calibration/joint_limits.json` and the shared `utils/hand_config.py`.

- `finger_control.py` — interactive CLI (`open`/`close`, `move <joint> <pct>`, `pose <name>`, `wrist`, `torque`).
- `joint_slider.py` — Tkinter slider per joint with live readback.
- `demo_gestures.py` — cycles named poses (`flat fist pinch point peace ok thumbs_up rock spider`).
- `grasp_control.py` — 200 Hz terminal teleop of grasp aperture; `retargeting.py` — MediaPipe hand-tracking from a webcam; `teleop_avp.py` — Apple Vision Pro bridge (publishes targets over ZMQ, received by `bringup.py`).
- `teach_and_repeat.py` — kinesthetic record/replay; `animations.py` — timestamped pose playback.
- `stress_test.py` — thermal cycling (stops if any motor exceeds 70 °C); `utils/data_visualization.py` — live 200 Hz telemetry plots + CSV log.

These remain percent-/script-oriented and are scheduled to migrate onto `orcahand` (which makes them a few lines each against `OrcaHand`); the hardware layer and the ROS node are the parts already unified.

6 Repository layout

```
firmware/orca_hand/
  orcahand/              # the shared hardware layer (see orcahand_api.pdf)
    orcahand/            # OrcaHand, FeetechBus, config, calibration, mock
    configs/             # canonical left/right yaml + calibration json
    tests/  examples/
  feetech/               # workbench firmware (this document)
    setup/               # precheck, setup_motor_chain, configure_registers,
    latency_test
    calibration/         # tension, calibrate_limits, smart_/manual_calibration
    control/             # finger_control, joint_slider, demo_gestures, grasp_control,
                         #   retargeting, teleop_avp, teach_and_repeat, stress_test,
    ...
    gui/                 # Tkinter dashboard (HandBus + panels)
    utils/hand_config.py # legacy shared constants (joint map, poses, pct<->raw)
    data/                # calibration/, gestures/, animations/, logs/
    models/              # MJCF + URDF of the hand
  orca_core/             # upstream reference library (Dynamixel + Feetech)
mabel_ws/src/mabel_orca_hand/   # ROS 2 node, wraps orcahand
```

7 Limitations and gotchas

- **Single bus, single point of failure.** All 17 servos share one TTL line; a single bad connector drops the whole hand. `precheck.py` / `orcahand-scan` isolate which IDs are missing.
- **Calibration is mechanical state.** Re-tensioning or a swapped servo invalidates `joint_limits.json`; stale limits silently shrink or shift the reachable range. Treat calibration as part of every assembly change.
- **Two identical adapters.** Auto-detect cannot distinguish two CH340s; for both hands, pin ports by `/dev/serial/by-id/`.
- **Percent vs. radians.** Legacy scripts use a 0–100 % convention (0 = open, 100 = closed); the API and ROS use radians. `orcahand` keeps `set_joint_percent` as a bridge, but radians is canonical.
- **Thermal.** Sustained high-load holds heat the HLS3915s; the stress test caps at 70 °C. Watch `get_state().temperature` / `HandState.hottest()` in long runs.

References

[1] FEETECH, *STS/SCS Serial Bus Servo Communication Protocol* and *HLS3915 / HLS3930 datasheets*.

[2] MABEL Robotics, *orcahand: A Clean Hardware-Abstraction Layer for the MABEL ORCA Dexterous Hand* (companion, `docs/orcahand_api.pdf`).

[3] ORCA Hand, *orca_core control library*, https://github.com/orcahand.